Jednokierunkowa wielowarstwowa sieć neuronowa typu MLP

Celem ćwiczenia jest zaprojektowanie i zbadanie zdolności uogólniania sieci MLP w klasyfikacji danych. Sieć jest uczona z wykorzystaniem wstecznej propagacji błędu. Do ćwiczenia należy opracować własny kod oraz uzasadnić przyjęte rozwiązania.

Środowisko i biblioteki

Środowisko implementacji może być dowolne. W przypadku języka Python należy wykorzystać następujące biblioteki:

- `numpy`: operacje numeryczne i identyfikacja typów danych;
- `pandas`: wczytanie danych i zapis wyników;
- `scikit-learn`: przygotowanie danych, walidacja krzyżowa, model MLP i metryki;
- `matplotlib`: przygotowanie wykresów wyników.

Wymagane biblioteki są zainstalowane na indywidualnych kontach.

Zakres badań

Zdolność uogólniania należy wyznaczyć za pomocą k-krotnej walidacji krzyżowej, powtórzonej n razy, gdzie k oznacza liczbę części (foldów), a n liczbę powtórzeń całej procedury. W każdym powtórzeniu należy zastosować nowy losowy podział na k części, przy czym wszystkie porównywane konfiguracje muszą być oceniane na dokładnie tych samych podziałach danych. Dla każdego podziału i każdej konfiguracji należy utworzyć oraz nauczyć nową sieć. Wartości k i n trzeba ustalić przed rozpoczęciem badań oraz podać w raporcie. Podstawowym kryterium pozostaje dokładność. Balanced accuracy i macro F1 mogą uzupełniać ocenę, zwłaszcza w przypadku nierównych liczności klas.

Analiza powinna uwzględniać:

1. sieci z jedną i dwiema warstwami ukrytymi oraz różną liczbą neuronów;
2. różne funkcje aktywacji w warstwach ukrytych;
3. różne algorytmy optymalizacji parametrów sieci;
4. średnią i odchylenie standardowe wartości $\text{Accuracy}_{\text{CV}}$ uzyskanych w n powtórzeniach k-krotnej walidacji krzyżowej;
5. opcjonalnie czas obliczeń.

Tabela 1: Odpowiedniki funkcji aktywacji w środowisku MATLAB i języku Python

Funkcja aktywacji	**MATLAB**	**Python – scikit-learn**
sigmoidalna	`logsig`	`logistic`
tangens hiperboliczny	`tansig`	`tanh`
liniowa	`purelin`	`identity`
ReLU	`poslin`	`relu`

W klasie `MLPClassifier` wybrana funkcja aktywacji jest stosowana we wszystkich warstwach ukrytych. Nie można przypisać różnych funkcji aktywacji poszczególnym warstwom.

W `scikit-learn` dostępne są algorytmy L-BFGS, SGD - opcjonalnie z momentum oraz Adam. Nie są one pełnymi odpowiednikami wszystkich metod dostępnych w MATLAB-ie. Algorytm SCG (`trainscg`) jest dostępny w MATLAB-ie, natomiast nie jest bezpośrednio dostępny w klasie `MLPClassifier`. Klasa ta nie udostępnia również algorytmu Levenberga-Marquardta, dostępnego w MATLAB-ie jako `trainlm`.

Organizacja podziałów i zapobieganie wyciekowi danych

Indeksy wszystkich $n \cdot k$ podziałów należy wygenerować jeden raz. Dla danego powtórzenia i foldu wszystkie porównywane konfiguracje muszą otrzymać dokładnie te same rekordy uczące i walidacyjne. Losową inicjalizację sieci również należy kontrolować: dla danego podziału porównywane konfiguracje powinny korzystać z tego samego ziarna inicjalizacji modelu.

Przykładowe przygotowanie podziałów w języku Python przedstawiono poniżej:

```
from sklearn.model_selection import RepeatedStratifiedKFold
k, n = 5, 10
cv = RepeatedStratifiedKFold(n_splits=k, n_repeats=n, random_state=42)
splits = list(cv.split(X, y))
for split_id, (train_idx, val_idx) in enumerate(splits):
    ...
```

Przed rozpoczęciem badań należy określić, czy standaryzacja danych jest potrzebna, oraz uzasadnić przyjętą decyzję.

Dla każdego podziału należy:

1. wydzielić nieprzetworzone dane uczące i walidacyjne;
2. jeżeli jest to potrzebne, dopasować standaryzację oraz inne operacje zależne od danych, np. uzupełnianie braków lub kodowanie cech, wyłącznie na części uczącej;
3. przekształcić część uczącą i walidacyjną przy użyciu parametrów wyznaczonych na danych uczących;
4. utworzyć i nauczyć oddzielny model dla każdej porównywanej konfiguracji, a następnie ocenić go na tej samej części walidacyjnej.

Jeżeli standaryzacja jest stosowana, jej parametrów nie wolno wyznaczać na całym zbiorze przed walidacją krzyżową. W każdym foldzie należy utworzyć nowy scaler, dopasować go wyłącznie do części uczącej, a następnie zastosować do części uczącej i walidacyjnej:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_val   = scaler.transform(X_val)
```

Tak przygotowany podział można wykorzystać we wszystkich konfiguracjach wymagających tego samego sposobu przetwarzania.

Uczenie modelu przez kolejne epoki

W języku Python można wykorzystać klasę `MLPClassifier`. Wszystkie parametry niebędące przedmiotem porównania powinny pozostać stałe; można przyjąć wartości domyślne, ale należy je podać.
Domyślnie `batch_size='auto'` oznacza $\min(200, N_{\text{train}})$, `learning_rate_init=0.001`, `alpha=0.0001` oraz `shuffle=True`. Dla `solver='sgd'` stosowane są także `momentum=0.9` i `nesterovs_momentum=True`. Podstawowy SGD bez momentum wymaga zmiany tych wartości na odpowiednio `0.0` i `False`.
Do kontrolowania uczenia w kolejnych epokach można wykorzystać metodę `partial_fit`. Przykładowy schemat przedstawiono poniżej:

```
classes = np.unique(y)

model = MLPClassifier(
        hidden_layer_sizes=(20, 10),
        activation="relu",
        solver="adam",
        batch_size="auto",
        learning_rate_init=0.001,
        alpha=0.0001,
        shuffle=True,
        random_state=model_seed)

for epoch in range(EPOCHS):
        if epoch == 0:
                model.partial_fit(X_train, y_train, classes=classes)
        else:
                model.partial_fit(X_train, y_train)
```

Jedno wywołanie metody `partial_fit` wykonuje jedną iterację uczenia na przekazanym zbiorze. Kolejne wywołania aktualizują ten sam model, dzięki czemu pętla odpowiada uczeniu jednej sieci przez ustaloną liczbę epok.
Argument `classes` zawiera pełny zbiór możliwych etykiet klas, np. `[0,1,...,25]`. Jest on wymagany podczas pierwszego wywołania `partial_fit`, ponieważ model musi znać liczbę i etykiety wszystkich klas. Informacja ta zostaje zapisana w modelu, dlatego w kolejnych epokach argument można pominąć.

Prezentacja wyników

Wyniki należy zaprezentować w następujący sposób:

- wykresy zmian średniej wartości $\text{Accuracy}_{\text{CV}}$ oraz jej odchylenia standardowego względem liczby epok; dla każdej epoki wartość $\text{Accuracy}_{\text{CV}}$ w jednym powtórzeniu stanowi średnia dokładność z k foldów, a statystyki wyznacza się na podstawie n powtórzeń; w ramach jednego eksperymentu porównywane warianty należy umieścić na wspólnym wykresie;
- boxploty wartości $\text{Accuracy}_{\text{CV}}$ uzyskanych po ustalonej liczbie epok dla porównywanych funkcji aktywacji, algorytmów optymalizacji i architektur; każdy wariant jest reprezentowany przez n wartości;
- dodatkowo można przygotować analogiczne wykresy zmian balanced accuracy i macro F1 względem liczby epok oraz odpowiadające im boxploty;
- funkcje aktywacji, algorytmy optymalizacji i architektury należy porównywać w osobnych eksperymentach; dla architektur trzeba podawać pełną konfigurację, np. $(10, 20)$, a nie tylko sumę neuronów.

Do wygenerowanych wykresów należy dodać odpowiednie wnioski. Wyniki powinny być również zestawione w tabeli podsumowującej.

Problemy do uwzględnienia

- rozpoznanie cech ilościowych i kategorycznych oraz sposób ich kodowania;
- dobór liczby warstw i neuronów, funkcji aktywacji oraz algorytmu optymalizacji.

Opcjonalnie można przeprowadzić analizę czasu uczenia, przedstawić wyniki dla dodatkowych metryk oraz dokonać interpretacji macierzy pomyłek.

Raport z zadania

W raporcie należy opisać:

- przygotowanie danych: liczbę rekordów, liczbę cech i klas oraz rozkład klas wyrażony liczbą rekordów w każdej klasie;
- podstawy teoretyczne modelu, ze szczególnym uwzględnieniem elementów analizowanych w badaniach;
- protokół oceny i sposób strojenia;
- kryterium wyboru modelu.

Poprawność rozwiązania wymaga, aby wszystkie porównywane konfiguracje były oceniane na tych samych podziałach danych.

Dodatkowe zadanie

Jeżeli wynik najlepszego modelu ma być interpretowany jako końcowa ocena zdolności predykcyjnej, należy użyć niezależnego zbioru testowego. Wymaga to wcześniejszego podziału danych z zachowaniem proporcji klas na zbiór uczący, zawierający na przykład 80% lub 90% rekordów, oraz zbiór testowy, zawierający odpowiednio 20% lub 10% rekordów. Walidację krzyżową i wybór konfiguracji należy przeprowadzić wyłącznie na zbiorze uczącym.

Po wyborze konfiguracji wszystkie operacje przetwarzania danych należy ponownie dopasować na całym zbiorze uczącym, a następnie zastosować do zbioru uczącego i testowego. Model należy ponownie nauczyć na całym zbiorze uczącym i tylko raz ocenić na niezależnym zbiorze testowym.

W przeciwnym przypadku można pozostać przy powtarzanej walidacji krzyżowej. Należy jednak pamiętać, że jest to ocena porównawcza konfiguracji, a nie niezależny test końcowy.

Wskazówka dotycząca organizacji badań i zapisywania wyników

Dla uproszczenia w każdym eksperymencie można zmieniać tylko jeden element konfiguracji, pozostawiając pozostałe parametry bez zmian. Na przykład:

1. **Porównanie funkcji aktywacji**
   - stała architektura: $(20, 10)$;
   - stały algorytm optymalizacji: `adam`;
   - porównywane funkcje: `identity`, `logistic`, `tanh`, `relu`.
2. **Porównanie algorytmów optymalizacji**
   - stała architektura: $(20, 10)$;
   - stała funkcja aktywacji: `relu`;
   - porównywane algorytmy: `adam`, `sgd`.
3. **Porównanie architektur**
   - stała funkcja aktywacji: `relu`;
   - stały algorytm optymalizacji: `adam`;
   - porównywane architektury: np. (20), $(20, 10)$, $(20, 20)$.

Dla każdego wariantu tworzona jest nowa sieć, uczona przez taką samą liczbę epok. Wszystkie warianty powinny wykorzystywać te same podziały danych, tę samą sekwencję ziaren inicjalizacji oraz te same zasady przetwarzania. Wyniki każdego z trzech eksperymentów można zapisać w oddzielnym pliku. Takie rozwiązanie ułatwia ocenę wpływu pojedynczego parametru, ale nie pozwala ocenić interakcji między parametrami.